

IST604 — Web Services and Distributed Computing

CA Test — June 2025 — Model Answers

University of Buea — Department of Computer Science

Course: IST604 – Web Services and Distributed Computing

Instruction: Answer all questions (30 marks total)

Question 1 — Distributed Computing (10 marks)

(a) One statement capturing the essence of distributed computing (1 mark)

Distributed computing is the method of making multiple computers work together as one coordinated system so that software components spread across networked machines communicate and cooperate to solve a common problem.

(Alternative acceptable wording: components of a software system are shared among multiple computers/nodes and run as one system, even though they are in different locations.)

(b) Benefits: Performance, Resilience and redundancy, Reuse (6 marks — ~2 marks each)

Performance

- Work is **split and executed in parallel** across multiple computers/nodes.
- Different parts of a task run **simultaneously**, so overall execution time and throughput improve.
- Load can be **distributed across several servers** instead of overloading one machine.

Resilience and redundancy

- **Multiple computers** can provide the **same service**.
- If one machine fails or is unavailable, **another can take over**, so the system keeps operating.
- If duplicate services run in **different data centres** and one site goes down, the organisation can still operate.

Reuse

- Distributed components can expose **services used by many client applications**.
 - This **reduces repeated development** and improves **interoperability** between components.
 - Shared network-accessible resources mean functionality is built once and reused widely.
-

(c) CORBA, DCOM, Java RMI — long form (3 marks — ~1 mark each)

CORBA (Common Object Request Broker Architecture)

CORBA is a standard from the **Object Management Group (OMG)** for distributed object computing. It is **language-neutral**: clients and servers can be written in different languages (e.g. a C++ client calling a Java server). It uses an **object request broker** so remote resources are treated as objects. It targets heterogeneous environments (different OS, languages, hardware). It was designed to overcome limitations of simple RPC in large, mixed environments.

DCOM (Distributed Component Object Model)

DCOM is a **Microsoft** technology for distributed computing on **Windows**. It lets Windows software components communicate over a network in a way similar to CORBA's object model, but it is **platform-specific** and optimised for **Windows-to-Windows** communication. Like CORBA, it uses a **synchronous, tightly coupled** RPC-style model. It did not become the dominant standard for large-scale Internet traffic partly because of **firewall and scalability** limitations compared with HTTP-based approaches.

Java RMI (Java Remote Method Invocation)

Java RMI is Sun's mechanism for **distributed Java objects**. It is **Java-only** and integrates with the JVM (garbage collection, security). Remote Java objects can be invoked and passed as arguments or return values as if they were local. It uses **JRMP** (Java Remote Method Protocol) between JVMs. Standard **java.net** APIs were not designed for full distributed-object challenges; RMI provides transparent remote method calls between Java applications on different machines.

Question 2 — Web Services (10 marks)

(a) What web services are used for (1 mark)

Web services are used to let different applications—on different platforms and in different languages—communicate and exchange data over a network using standard protocols (e.g. HTTP), so systems can integrate, share functionality, and interoperate without tight coupling to a single vendor or technology.

(b) Interoperability, loose coupling, re-usability (3 marks — ~1 mark each)

Interoperability

Applications built in **different languages** and on **different platforms** (e.g. Java and .NET) can communicate through a web service because messages use **standard formats** (XML/JSON) and **standard protocols** (HTTP). Each side does not need to know the other's internal implementation.

Loose coupling

The **client and server are independent**: changes inside one system (e.g. database or business logic) do not force changes in the other, as long as the **published interface/contract** stays the same. This improves maintainability and allows services to be updated or replaced with less impact on consumers.

Re-usability

A business function (e.g. payment, authentication) is implemented **once** as a web service and **reused** by many applications over the network. This avoids duplicate coding and keeps behaviour consistent across the organisation or partners.

(c) Roles: Registry, Service Provider, Service Consumer (3 marks — ~1 mark each)

Component	Role
Registry	A directory (often implemented with UDDI) where services are registered and discovered . Providers publish service descriptions; consumers search the registry to find services.
Service Provider	Develops, deploys, and hosts the web service. Publishes the service interface/description (e.g. WSDL) to the registry and responds to invocations.
Service Consumer (Requester)	Finds the service (via registry), binds to the provider using the service description, and invokes operations (e.g. sends SOAP requests or REST HTTP calls).

Typical flow: Provider **publishes** → Consumer **finds** (registry) → Consumer **binds and invokes** (provider).

(d) SOAP vs REST — three points (3 marks — ~1 mark each)

Aspect	SOAP Web Services	REST Web Services
Message format	XML only — every message is a SOAP Envelope (Header, Body, optional Fault). Verbose but strictly structured.	Flexible — commonly JSON ; also XML, HTML, text. No mandatory envelope.
Service description	WSDL required — formal XML contract describing operations, types, bindings, and endpoint URL.	Optional — often OpenAPI/Swagger or informal documentation; no single mandatory standard like WSDL.
Protocol used	Transport-neutral in theory (HTTP, SMTP, JMS, etc.); HTTP is most common.	HTTP/HTTPS only — uses GET, POST, PUT, DELETE on resources (URLs).

Question 3 — SOAP (10 marks)

(a) Abbreviations (3 marks — 1 mark each)

Abbreviation	Stands for
SOAP	Simple Object Access Protocol
UDDI	Universal Description, Discovery, and Integration

Abbreviation	Stands for
WSDL	Web Services Description Language

(b) SOAP elements: Envelope, Header, Body — roles and optional/obligatory (3 marks)

Element	Role	Required?
Envelope	Root element of every SOAP message; identifies the document as SOAP and wraps all other SOAP elements.	Obligatory (required)
Header	Optional metadata for the message — e.g. authentication, routing, transactions, security tokens. Processed by intermediaries or the ultimate receiver as directed.	Optional
Body	Contains the actual payload — the request or response data (operation input/output).	Obligatory (required)

Note: Fault (error information) is optional and appears **inside the Body** when an error occurs.

Summary: Envelope and **Body** are required; **Header** is optional.

(c) RPC Style vs Document Style SOAP (4 marks)

RPC Style

- SOAP Body is structured like a **remote procedure call**: wrapper element named after the **operation**, with **child elements for each parameter**.
- Maps closely to **method name + parameters**.
- Historically used for **simple types** (String, int, etc.).
- In JAX-WS: `@SOAPBinding(style = SOAPBinding.Style.RPC)`.

Document Style

- SOAP Body contains a **complete XML document** as the message (not a direct parameter list per method).
- Better for **complex/rich data** and extensible messages.
- **Industry standard** and **JAX-WS default** (if `@SOAPBinding` is omitted, Document style is used).
- In JAX-WS: `@SOAPBinding(style = SOAPBinding.Style.DOCUMENT)` or omit annotation.

Main difference (exam summary)

	RPC Style	Document Style
Structure	Operation wrapper + parameters	Self-contained XML document
Data	Simpler types	Complex types, documents
Status	Older/simpler	Preferred default today

End of CA model answers — June 2025.